



Java : Notions de base

■ Généralités

- Langage développé par Sun Microsystems en 1990 et lancé en 1995 racheté par Oracle en 2009
- S'apparente beaucoup au C++
- Editions:
 - ✓ Java SE qui permet le développement d'applications standard
 - ✓ Java EE qui permet le développement d'applications d'entreprise
 - ✓ Java ME qui permet le développement d'applications mobiles



Java : Notions de base

■ Objets et classes

- Un objet possède 2 aspects :
 - ✓ Il est doté d'une identité (une référence) qui s'oppose à la notion de valeur
 - ✓ Il est composé accessoirement de données et de procédures (méthodes) liées aux données
- Une classe est un type ou modèle d'objets
- Définition de classe
 - ✓ `[public] [abstract] class nom_de_la_classe`
`{ ... }`
 - ✓ `abstract` signifie que la classe n'est pas encore terminée



Java : Notions de base

■ Objets et classes

- Définition de classe : Exemple



```
class Rectangle {  
    private float L; private float l;  
    public Rectangle(float longueur, float largeur)  
        { L=longueur; l=largeur;}  
    public Rectangle()  
        {L=0; l=0;}  
    public float perimetre()  
        {return (2*(L+l));}  
    public float surface()  
        { return (L*l);}  
}
```



Java : Notions de base

■ Objets et classes

- Définition de classe

- ✓ Les données constantes sont définies à l'aide du mot clé **final**
- ✓ Les composants d'une classe peuvent être qualifiés de:
 - **private** : accessible uniquement par les membres de la classe
 - **protected** : accessible par les classes héritières, par les classes du même paquetage et par les membres de la classe
 - **public** : accessible par tous
 - Si le composant n'est pas qualifié, il est accessible par les classes du même paquetage
- ✓ Une classe peut définir un ou plusieurs constructeurs
- ✓ l'instance courante est référé par le mot clé **this**
- ✓ Un composant non lié à des objets est qualifié de **static**

Java : Notions de base

■ Objets et classes

- Déclaration et création d'objets

- ✓ Syntaxe de déclaration

- **nom_de_classe nom_objet1 [,nom_objet2 ...]**
- Exemple : `Rectangle rect1, rect2, rect3`
- La déclaration définit un identificateur et lui associe un type

- ✓ Syntaxe de création

- **[accès] [static] [final] nom_objet = new nom_de_classe(paramètres)**
accès peut être : private, protected ou public
- Exemple : `rect1 = new Rectangle(5.2,3.0);`
`rect2 = new Rectangle();`

- ✓ Utilisation des objets

- `rect1.surface(); rect2.perimetre();` mais `rect3.L=12.0;` est interdit



Java : Notions de base

■ Objets et classes

- Classes internes

- ✓ Classe déclarée à l'intérieur d'une classe (inner class). Elle peut être déclarée **private** (donc pas visible de l'extérieur)
- ✓ Une classe interne peut être accédée de l'extérieur de la manière suivante: ***classe_englobante.classe_interne***
- ✓ Si la classe interne ne fait référence à aucun membre de la classe englobante on peut la déclarer **static**
- ✓ Déclaration et création d'un objet de la classe interne:
 - ***classe_englobante.classe_interne objet_classe_interne = objet_classe_englobante.new constructeur_classe_interne***



Java : Notions de base

■ Éléments du langage

- Types de données
 - ✓ Entier : `byte` – `short` – `int` – `long`
 - ✓ Réel : `float` – `double`
 - ✓ Caractère : `char`
 - ✓ Booléen : `boolean`
 - ✓ Chaîne de caractères : `String` opérateur `+` pour la concaténation
 - ✓ Null : `null` constante pour créer objet pas encore utile
 - ✓ Tableau :
 - déclaration *`type [] nom_tab` ou `type nom_tab []`*
 - création *`type nom_tab [] = new type [taille]`*
 - Taille du tableau : *`nom_tab.length`*



Java : Notions de base

■ Éléments du langage

- Opérateurs

- ✓ arithmétique : idem que C/C++
- ✓ logique : idem que C/C++
- ✓ binaire : idem que C/C++
- ✓ conversion :
 - Ad hoc : conversion automatique de format lors d'affectation
 - Casting : conversion d'un type en un autre
`type1 valeurType1 = (type1) valeurType2`
 - À l'aide de méthode
 - toxxx par exemple toString
 - xxxValue par exemple longValue
 - parsexxx par exemple parseInt



Java : Notions de base

■ Éléments du langage

- Structures de contrôle
 - ✓ idem que C/C++
- Commentaires
 - ✓ // sur une ligne
 - ✓ /* et */ sur plusieurs lignes
 - ✓ /** et */ commentaire transformable par Javadoc pour produire de la documentation HTML
- Gestion des erreurs : 2 manières
 - ✓ Propager via la directive **throws** : *type nom_méthode() throws nom_exception { ...}*



Java : Notions de base

■ Éléments du langage

- Gestion des erreurs : 2 manières
 - ✓ Capter et traiter à l'aide de routines try – catch au sein de méthode
 - *type méthode1 () { ...
 try { ... méthode2 () ...}
 catch (nom_exception e) {traitement de e}
}*
 - on peut ajouter la routine **finally** { ... } qui s'exécutera dans tous les cas erreur ou pas.

Java : Notions de base

■ Programme Java

- Structure d'une application

```
import *.* ; import ... ;

public class Exemple
{
    //Déclaration des donnés
    // déclaration des méthodes
    [accès] [static] type méthodes (paramètres)
    {
        "Déclaration de variable locales
        .....
    }
    .....
}
```

- ✓ Une seule classe public qui doit avoir le même nom que le fichier programme java. Ici fichier Exemple.java

- ✓ **import** comme **#include** en C/C++



Java : Notions de base

■ Programme Java

- ✓ Si une des méthodes de la classe publique du fichier est main.
L'application est interprétable par java
 - Le profil de la méthode main doit être :
`public static void main(String argv[])`
- ✓ Si la classe publique dérive de Applet la classe est exécutable dans un navigateur
- ✓ Dans les autres cas, la classe est utilisable par une autre classe
- Paquetages
 - ✓ Les classes sont regroupées en paquetages
 - ✓ Création d'un paquetage par : `package nom_paquetage;`
 - ✓ Utilisation des classes d'un paquetage par : `import paquetage.*`



Java : Notions de base

■ Programme Java

- Entrées/Sorties
 - ✓ Sortie sur écran utilise méthodes statique
 - *System.out.print(String s) ou System.out.println(String s)*
 - String s peut être une concaténation de valeurs numériques et chaînes de caractères
 - ✓ Entrée au clavier utilise méthode statique
 - *System.in.read()* : lecture d'un caractère
 - Pour lire un nombre ou chaîne de caractères, on lit plusieurs caractères et on convertit dans le type approprié
 - La classe Lire contient des méthodes pour lire : une chaîne Lire.S(), un entier Lire.i(), un byte Lire.b(), un short Lire.s(), un long Lire.l(), un float Lire.f(), un double Lire.d(), un caractère Lire.c()
 - La classe Lire est disponible sous fichier java



Java : Notions de base

■ Programme Java

- Entrées/Sorties sur fichiers
 - ✓ Fichiers textes utilisent les classes `FileReader` et `FileWriter`
 - Syntaxes de déclaration:
 - *`FileReader entrée = new FileReader(new File("fichier.txt"))` ou `FileReader entrée = new FileReader("fichier.txt")`*
 - Les méthodes disponibles : `read()` et `write()`
 - ✓ On peut utiliser les classes `BufferedReader` et `BufferedWriter`
 - Syntaxes :
 - `BufferedReader br = new BufferedReader (new InputStreamReader (System.in)); String chaine= br.ReadLine();`* pour lecture au clavier
 - `BufferedReader fr = new BufferedReader (new FileReader (new File(nom_fichier))); String chaine=fr.ReadLine();`* pour lecture dans un fichier



Java : Notions de base

■ Travaux pratiques

- Fiche de TD exercices 1, 2 et 3

Java : Notions de base

■ Héritage

- Définir une classe similaire à une classe existante, avec des propriétés supplémentaires
 - ✓ Syntaxe : `class Classe_fille extends Classe_mere { ... }`
 - ✓ Exemple :
 - ```
class Personne {
 String nom; int age;
 Personne (String lenom, int lage){nom=lenom;age=lage;}
}
```
    - ```
class Salarie extends Personne{  
    int salaire; int prime(){ return salaire*age/100;}  
    Salarie(String n, int a, int s){ super(n,a);salaire=s;}  
}
```


Java : Notions de base

■ Héritage

- ✓ ***super(paramètres)*** dans un constructeur fait appel au constructeur de la classe mère
- ✓ Quand une méthode est redéfinie dans la classe fille, l'héritage est supprimé on appelle la méthode de la classe mère par ***super.méthode_classe_mère***
- ✓ Une méthode sans l'attribut ***final*** peut être redéfinie
- ✓ Une classe avec une méthode non définie est une classe abstraite, elle est non instanciable :

```
■ abstract class A {  
    ...  
    abstract void p();  
    ...  
}
```



Java : Notions de base

■ Interface – héritage multiple

- Pas d'héritage multiple en Java mais interface
- Interface :
 - ✓ Collection de déclaration de méthodes
 - ✓ Ne définit aucun corps de méthode, ni aucun composant variable (cas limite de classe abstraite)
 - ✓ Ne figure que profil de méthodes et éventuellement déclaration de constantes
syntaxe : ***[public] interface Nom_Interface { ... }***
 - ✓ Une classe peut implémenter une ou +sieurs interfaces
 - ✓ Syntaxe : ***class Nom_classe implements
Nom_interface1 [Nom_interface2 ...] { ... }***



Java : Notions de base

■ Travaux pratiques

- Fiche de TD exercices 4 et 5